Universidade de São Paulo
ACH2027 - Gestão de Projetos de Tecnologia da Informação
Prof. Doutor: Edmir Parada Vasques Prado **Ano:** 2º Semestre/2026

Grupo 7 - AHP: escolha da ferramenta do trabalho semestral

Gustavo G. França do Nascimento
15637551
T94

Kevin Rodrigues Nunes
15676030
T94

Kauã Lima de Souza
15674702
T94

Victor Yodono
13829040
T94

I. Objetivo e hierarquia

O objetivo é escolher, com o AHP (Analytic Hierarchy Process), a ferramenta para implementar o trabalho semestral: o Sistema de Controle de Acesso da EACH, que controla a entrada e a saída de pessoas e veículos, libera automaticamente a comunidade pelo Nº USP ou CPF, cadastra visitantes, tem hierarquia de acesso por senha, faz o pré-cadastro em lote para grandes eventos, emite relatórios de frequência e se integra às catracas e cancelas. A escolha é o trabalho da atividade K, "Definir a stack tecnológica" (EAP 1.2.2.1, no pacote Documento de arquitetura), do cronograma do grupo (Exercício 2). As quatro alternativas e os quatro critérios foram definidos pelo professor, por e-mail, em 5 de outubro de 2026.

O AHP organiza a decisão em uma hierarquia, com o objetivo no topo, os critérios no meio e as alternativas na base (Figura I) [31,47]. É um caso de análise multicritério ponderada, que o PMBOK cita como exemplo de esquema de priorização [26, p. 181].

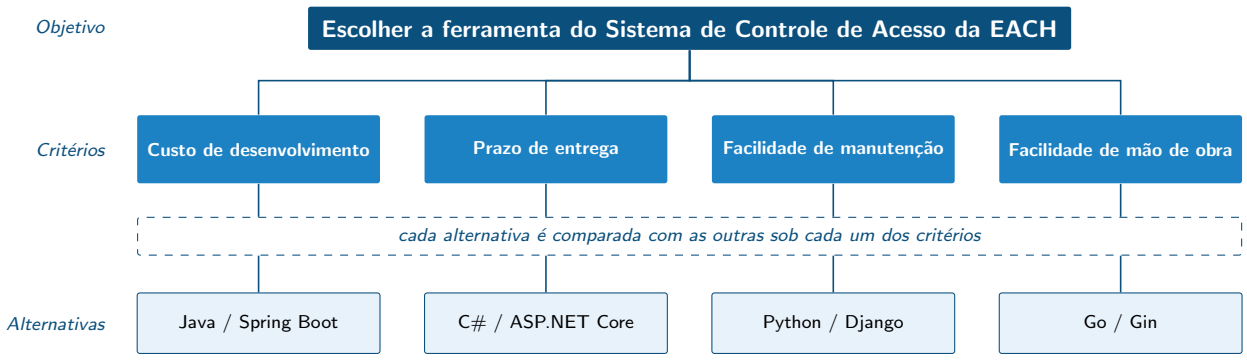


Figura I: Hierarquia da decisão.

II. Premissas e critérios

- A equipe é a de bolsistas: 5 a 8 alunos do último ano de SI, com o Analista de Sistemas Sênior. Os Exercícios 1 e 2 deixaram em aberto a escolha entre bolsistas e uma start-up, e o resultado vale para a equipe de bolsistas.
- Cada alternativa é a linguagem com o seu principal framework web: Java/Spring Boot, C#/ASP.NET Core, Python/Django e Go/Gin.

A Tabela I define os critérios. Cada fator entra em um único critério, para não ser contado duas vezes: a curva de aprendizado, por exemplo, só pesa na mão de obra, porque o prazo já supõe um time que sabe a linguagem.

Critério	O que mede	Base
Custo de desenvolvimento	Gastos em R$ que mudam com a linguagem: licenças, ferramentas, hospedagem e integração com catracas e cancelas. As bolsas são fixas e ficam de fora.	PMBOK 6ª ed., cap. 7 [27]
Prazo de entrega	O que o framework traz pronto para entregar os quatro módulos (controle de acesso, gestão de usuários e visitantes, rotinas especiais e relatórios), supondo um time que já sabe a linguagem.	PMBOK 6ª ed., cap. 6 [27]
Facilidade de manutenção	Depois da implantação (a manutenção ficou fora do escopo do projeto, Exercício 1), quão fácil é para quem herdar o código corrigir, evoluir e adaptar o sistema. A norma define a manutenibilidade como "Capability of a product to be modified by the intended maintainers with effectiveness and efficiency". A norma inclui aí a instalação de atualizações ("Maintainability includes installation of updates and upgrades"), e por isso o tempo de suporte das versões entra neste critério.	ISO/IEC 25010:2023 [2]
Facilidade de mão de obra	Durante o projeto, achar bolsistas de SI que já sabem a linguagem, a curva de aprendizado de quem não sabe e a reposição de quem sair.	PMBOK 6ª ed., 9.3 Adquirir Recursos: critérios de seleção como disponibilidade, experiência e conhecimento [27]

Tabela I: Definição dos critérios.

III. Método

Em cada matriz, o elemento da linha é comparado ao da coluna pela escala fundamental de Saaty (Tabela II) [31]. A comparação inversa recebe o recíproco: se Java vale 3 diante de Go, Go vale 1/3 diante de Java.

Valor	Significado
1	Igual importância
3	Importância moderada
5	Importância essencial ou forte
7	Importância muito forte
9	Importância extrema
2, 4, 6, 8	Valores intermediários entre dois julgamentos vizinhos
1/2 a 1/9	Recíprocos, para a comparação inversa

Tabela II: Escala fundamental de Saaty.

O vetor de prioridades de cada matriz é o seu autovetor principal [31]. A ideia do cálculo aparece na aproximação usada por Vargas [47]: dividir cada valor pela soma da sua coluna e tirar a média de cada linha. Essa média aproxima a prioridade de cada elemento; as tabelas trazem o autovetor exato, calculado por programa, e as prioridades de cada matriz somam 100%.

Os julgamentos podem se contradizer: se Java vence C# e C# vence Go, Go não deveria vencer Java. A consistência é medida pelo autovalor principal $\lambda_{\max}$, que é igual a n, o número de elementos comparados, quando a matriz é perfeitamente consistente [31]:

$$\mathrm{CI} = \frac{\lambda_{\max} - n}{n - 1} \qquad \mathrm{CR} = \frac{\mathrm{CI}}{\mathrm{RI}}$$

O CI é o índice de consistência, e o CR (Consistency Ratio) o compara ao índice aleatório RI, que vale 0,90 para $n = 4$. A matriz é consistente quando $\mathrm{CR} < 0{,}10$ [47].

A pontuação global de cada alternativa a soma, sobre os critérios c, o peso do critério vezes a prioridade da alternativa naquele critério [47]:

$$G(a) = \sum_{c} w_c \, p_c(a)$$

em que w_c é o peso do critério c e $p_c(a)$ é a prioridade de a sob c.

IV. Pesos dos critérios

A Tabela III compara os critérios dois a dois quanto à importância para o objetivo. Os pesos ficam em 7,0% para custo de desenvolvimento, 36,8% para prazo de entrega, 19,3% para facilidade de manutenção e 36,8% para facilidade de mão de obra, com CR de 0,002.

Prazo de entrega e facilidade de mão de obra empatam como os critérios mais pesados, porque a equipe só existe enquanto duram as bolsas. No cronograma do Exercício 2, as bolsas cobrem cerca de 26 semanas a partir da formalização da equipe, da semana 10 à 35; o projeto termina na semana 26, e as 9 semanas restantes são a única reserva para atrasos. Uma ferramenta que atrase a entrega consome essa reserva, e uma equipe que não conheça a linguagem gasta parte dessas mesmas semanas aprendendo. A facilidade de manutenção vem em seguida: a manutenção ficou fora do escopo, mas o sistema continua em uso depois da implantação. O custo pesa pouco, porque as quatro linguagens e seus frameworks têm licenças de código aberto (Tabela IV) e as bolsas são fixas.

	Custo de desenvolvimento	**Prazo de entrega**	**Facilidade de manutenção**	**Facilidade de mão de obra**	**Prioridade**
Custo de desenvolvimento	1	1/5	1/3	1/5	7,0%
Prazo de entrega	5	1	2	1	36,8%
Facilidade de manutenção	3	1/2	1	1/2	19,3%
Facilidade de mão de obra	5	1	2	1	36,8%

Tabela III: Matriz de comparação dos critérios e vetor de pesos (CR = 0,002).

V. Alternativas em cada critério

Em cada critério, as quatro alternativas são comparadas duas a duas, sempre para a equipe de bolsistas descrita nas premissas. O PMBOK põe a escolha da ferramenta entre as decisões de grande impacto de um projeto e lembra que decidir em grupo aproveita o conhecimento de mais pessoas e aumenta o compromisso com o resultado [26, p. 27–28]. Por isso cada matriz vem com o motivo dos seus julgamentos e a fonte de cada fato, para que qualquer membro possa conferir e contestar um número.

A. Custo de desenvolvimento

	Java	**C#**	**Python**	**Go**	**Prioridade**
Java	1	1	1	1	25,0%
C#	1	1	1	1	25,0%
Python	1	1	1	1	25,0%
Go	1	1	1	1	25,0%

Tabela IV: Matriz de comparação e vetor de prioridades sob o critério custo de desenvolvimento (CR = 0,000).

A Tabela IV tem CR de 0,000. As quatro linguagens e seus frameworks têm licenças de código aberto: GPL com *Classpath Exception* no OpenJDK, Apache 2.0 no Spring Boot, MIT no .NET, no ASP.NET Core e no Gin, a licença da PSF no Python e licenças no estilo BSD no Django e no Go [3, 9, 15, 21, 25, 28, 35, 41]. A única ressalva está nas ferramentas do C#. Em organizações com mais de 250 computadores ou mais de US$ 1 milhão de receita anual, o Visual Studio Community e o C# Dev Kit só podem ser usados em projetos de código aberto, pesquisa acadêmica e sala de aula [17, 24], e a USP, com receita prevista de R$ 10,58 bilhões em 2026 [46], passa desse porte. Mesmo assim, a extensão C# do VS Code pode ser usada sozinha [16], e os seus termos não limitam o porte da organização [18]; o VS Code é gratuito também para uso comercial [23]. Sem o Dev Kit, perdem-se a visão de projeto e o explorador de testes, que vêm com as assinaturas pagas do Visual Studio [17]; este trabalho supõe que o time trabalhe sem eles. As fontes consultadas não diferenciam as quatro na hospedagem nem na integração com catracas e cancelas, e esses itens

ficaram iguais. Por isso as quatro empatam neste critério.

B. Prazo de entrega

	Java	**C#**	**Python**	**Go**	**Prioridade**
Java	1	1/2	1/3	3	16,2%
C#	2	1	1/2	5	28,8%
Python	3	2	1	7	49,1%
Go	1/3	1/5	1/7	1	6,0%

Tabela V: Matriz de comparação e vetor de prioridades sob o critério prazo de entrega (CR = 0,007).

A Tabela V tem CR de 0,007. O Django gera um painel administrativo a partir dos modelos, pensado para uso interno, e traz usuários, permissões, grupos e telas de login [5, 6]: isso cobre boa parte dos cadastros e da hierarquia de acesso. O ASP.NET Core Identity cuida de usuários, senhas, papéis e confirmação de e-mail e traz telas de login prontas [19]. O Spring Security traz autenticação, autorização por papéis e uma API para gerenciar usuários [32, 34, 37], mas não as telas. Os dois fazem o acesso ao banco pelo Spring Data JPA e pelo EF Core, com migrações do esquema [20, 33, 36], e nenhum traz o painel. A documentação do Gin cobre rotas, middleware, validação e renderização [10] e mostra o banco com o GORM, que também faz as migrações [8]; para usuários, o Gin só oferece um middleware de autenticação básica [12] e sessões por uma biblioteca à parte [11], sem papéis, e o cadastro e a hierarquia de acesso ficariam com o time. A ordem fica Python, pelo painel; C#, pelas telas de login; Java; e Go, pela falta de usuários e papéis.

C. Facilidade de manutenção

	Java	**C#**	**Python**	**Go**	**Prioridade**
Java	1	1/2	2	1	22,7%
C#	2	1	3	2	42,4%
Python	1/2	1/3	1	1/2	12,2%
Go	1	1/2	2	1	22,7%

Tabela VI: Matriz de comparação e vetor de prioridades sob o critério facilidade de manutenção (CR = 0,004).

A Tabela VI tem CR de 0,004. Dois fatores separam as alternativas. O primeiro é a tipagem: Java, C# e Go têm tipagem estática [13, 22, 43], e o compilador pega os erros de tipo antes de o código rodar [13, 22]; no Python, as anotações de tipo são opcionais e o interpretador não as verifica [30], e, sem um verificador externo, esses erros só aparecem na execução. O segundo é o tempo de suporte das versões, que entra na manutenção porque a norma inclui nela a instalação de atualizações [2]. O .NET tem versões LTS de três anos, que valem também para o ASP.NET Core [14]; o Django tem versões LTS de três anos [4], e o Python mantém cada versão por cinco [29]. No Java, o JDK tem versões LTS gratuitas de pelo menos quatro anos [7], mas cada versão menor do Spring Boot tem pelo menos 13 meses de suporte gratuito [38]; no Go, cada versão só recebe correções até saírem duas mais novas [42], ainda que a promessa de compatibilidade do Go 1 torne a troca mais simples [40]. O C# vai bem nos dois fatores e fica à frente. Java e Go empatam, com tipagem estática e atualizações do framework ou da linguagem a cada ano, mais ou menos. O Python tem o suporte mais longo, mas fica atrás, porque a tipagem pesou mais que o suporte.

D. Facilidade de mão de obra

	Java	**C#**	**Python**	**Go**	**Prioridade**
Java	1	3	2	5	48,3%
C#	1/3	1	1/2	2	15,7%
Python	1/2	2	1	3	27,2%
Go	1/5	1/2	1/3	1	8,8%

Tabela VII: Matriz de comparação e vetor de prioridades sob o critério facilidade de mão de obra (CR = 0,005).

A Tabela VII tem CR de 0,005. A ACH2003 (Computação Orientada a Objetos), cuja bibliografia é de Java [44], é obrigatória no 3º semestre da grade de SI publicada no JupiterWeb, que vale a partir de 2027; nas ementas dessa grade, o Python só aparece em duas optativas de computação sônica, e C# e Go não aparecem [45]. A versão atual da ACH2003 vale desde 2021 [44], mas a grade em que entraram os bolsistas de hoje não foi consultada. Fora da EACH, a pesquisa do Stack Overflow de 2025 põe o Python à frente das outras três, tanto entre desenvolvedores profissionais (54,8%, contra 29,9% do C#, 29,6% do Java e 17,4% do Go) quanto entre quem está aprendendo a programar (71,8%, contra 40,8% do Java, 23,1% do C# e 13,1% do Go) [39]. A pesquisa mede o mundo todo, não os alunos da EACH, e as intensidades entre Python, C# e Go seguem a aba de profissionais, onde as diferenças são menores. O Java fica à frente pela disciplina obrigatória, e o Python, em segundo, pela pesquisa. As fontes não medem a curva de aprendizado de cada linguagem, e a reposição de quem sair depende do mesmo fator: achar quem já conhece a linguagem.

VI. Ranking global

Na Tabela VIII, cada célula de critério é a prioridade da alternativa naquele critério, e o total é a soma ponderada pelos pesos do cabeçalho. Os percentuais têm uma casa decimal e podem não somar exatamente 100%. A ferramenta escolhida é Python, com pontuação global de 32,2%, à frente de Java, com 29,9%. O Python vence pelo prazo, graças ao Django, e o Java fica em segundo pela mão de obra; o C#, à frente na manutenção, fica atrás do Python no prazo e dos dois na mão de obra.

Posição	**Alternativa**	**Custo de desenvolvimento** (7,0%)	**Prazo de entrega** (36,8%)	**Facilidade de manutenção** (19,3%)	**Facilidade de mão de obra** (36,8%)	**Total**
1º	Python	25,0%	49,1%	12,2%	27,2%	32,2%
2º	Java	25,0%	16,2%	22,7%	48,3%	29,9%
3º	C#	25,0%	28,8%	42,4%	15,7%	26,3%
4º	Go	25,0%	6,0%	22,7%	8,8%	11,6%

Tabela VIII: Ranking global: prioridade de cada alternativa em cada critério e pontuação global.

Para medir a firmeza do resultado, cada um dos 30 julgamentos foi movido um ponto na escala, um de cada vez e para cada lado: um 3 vira 2 ou 4, e um 1 vira 2 a favor de um ou do outro elemento. O primeiro lugar muda em 3 das 60 mudanças (Tabela IX). O valor é o da célula linha × coluna, como nas matrizes: 1/2 quer dizer que a coluna passa a valer um pouco mais que a linha.

Matriz	**Linha × coluna**	**Valor**	**Novo ranking**
Critérios	Prazo de entrega × Facilidade de mão de obra	$1 \to 1/2$	Java (32,0%) à frente de Python (30,9%)
Prazo de entrega	C# × Python	$1/2 \to 1$	Java (30,0%) à frente de Python (29,7%)
Facilidade de mão de obra	Java × Python	$2 \to 3$	Java (31,4%) à frente de Python (31,0%)

Tabela IX: Mudanças de um ponto, em um único julgamento, que trocam o primeiro lugar.

VII. Registro de Uso de Inteligência Artificial

Em conformidade com a Política de Uso de Ferramentas de Inteligência Artificial Generativa da disciplina ACH2027 [1], declaramos nas Tabelas X e XI o uso de IA generativa neste documento.

Registro de Uso de IA	**Registro 01: Pesquisa de referências**	**Registro 02: Formatação em LaTeX e PDF**
Problema / Objetivo	Encontrar referências sobre os assuntos da atividade (o AHP, os critérios e as quatro linguagens com seus frameworks) para fundamentar o método e os julgamentos.	Formatar o documento no modelo LaTeX do grupo, criar o layout das tabelas e do diagrama da hierarquia e exportar o resultado para PDF.
Ferramenta de IA	Claude Opus 5.5, via Claude Code (Outubro/2026).	Claude Opus 5.5, via Claude Code (Outubro/2026).
Comandos	"veja na internet e nas refs que estamos usando, o PMBOK é bom".	"pode ser, fecha com PDF em atividades/ahp" e "crie a tabela e pode fazer o PDF com essa seção...".
Revisão dos resultados	[PREENCHER: revisão do grupo]	[PREENCHER: revisão do grupo]
Aplicação no trabalho	Citações das Seções I, II, III e V e a lista de Referências.	Todo o documento: a Figura I, as tabelas das Seções II a VI e este registro.

Tabela X: Registro de Uso de IA Generativa.

Registro de Uso de IA	**Registro 03: Vantagens e desvantagens de cada linguagem**
Problema / Objetivo	Pesquisar as vantagens e desvantagens de Java/Spring Boot, C#/ASP.NET Core, Python/Django e Go/Gin em cada critério: licenças e ferramentas gratuitas, recursos prontos de login e cadastro, prazo de suporte das versões, tipagem e oferta de quem conhece cada linguagem.
Ferramenta de IA	Claude Opus 5.5, via Claude Code (Outubro/2026).
Comandos	"faça esse: ... uma ficha com a definição de cada critério aplicada ao caso e os fatos de cada linguagem".
Revisão dos resultados	[PREENCHER: revisão do grupo]
Aplicação no trabalho	Seção V (os parágrafos que justificam cada matriz das alternativas).

Tabela XI: Registro de Uso de IA Generativa (continuação).

Referências

[1] ACH2027 - GESTÃO DE PROJETOS DE TECNOLOGIA DA INFORMAÇÃO. **Política de Uso de Ferramentas de Inteligência Artificial Generativa da Disciplina de Gestão de Projetos de TI (ACH2027)**: 2º semestre de 2026. São Paulo: EACH-USP, 2026.

[2] ARC42. **Maintainability**. Disponível em: `https://quality.arc42.org/qualities/maintainability`. Acesso em: 12 out. 2026. Reproduz a definição da ISO/IEC 25010:2023.

[3] DJANGO SOFTWARE FOUNDATION. **django/LICENSE**. GitHub. Disponível em: `https://github.com/django/django/blob/main/LICENSE`. Acesso em: 12 out. 2026.

[4] DJANGO SOFTWARE FOUNDATION. **Download Django**. Disponível em: `https://www.djangoproject.com/download/`. Acesso em: 12 out. 2026.

[5] DJANGO SOFTWARE FOUNDATION. **The Django admin site**. Disponível em: `https://docs.djangoproject.com/en/stable/ref/contrib/admin/`. Acesso em: 12 out. 2026.

[6] DJANGO SOFTWARE FOUNDATION. **User authentication in Django**. Disponível em: `https://docs.djangoproject.com/en/stable/topics/auth/`. Acesso em: 12 out. 2026.

[7] ECLIPSE FOUNDATION. **Temurin Support**. Adoptium. Disponível em: `https://adoptium.net/support/`. Acesso em: 12 out. 2026.

[8] GIN. **Database Integration**. Gin Web Framework. Disponível em: https://gin-gonic.com/en/docs/server-config/database/. Acesso em: 12 out. 2026.

[9] GIN. **gin/LICENSE**. GitHub. Disponível em: https://github.com/gin-gonic/gin/blob/master/LICENSE. Acesso em: 12 out. 2026.

[10] GIN. **Introduction**. Gin Web Framework. Disponível em: https://gin-gonic.com/en/docs/introduction/. Acesso em: 12 out. 2026.

[11] GIN. **Session Management**. Gin Web Framework. Disponível em: https://gin-gonic.com/en/docs/middleware/session-management/. Acesso em: 12 out. 2026.

[12] GIN. **Using BasicAuth middleware**. Gin Web Framework. Disponível em: https://gin-gonic.com/en/docs/middleware/using-basicauth-middleware/. Acesso em: 12 out. 2026.

[13] GOSLING, J. *et al.* **The Java Language Specification**: Java SE 25 Edition. Cap. 4, Types, Values, and Variables. Oracle. Disponível em: https://docs.oracle.com/javase/specs/jls/se25/html/jls-4.html. Acesso em: 12 out. 2026.

[14] MICROSOFT. **.NET and .NET Core support policy**. Disponível em: https://dotnet.microsoft.com/en-us/platform/support/policy/dotnet-core. Acesso em: 12 out. 2026.

[15] MICROSOFT. **aspnetcore/LICENSE.txt**. GitHub. Disponível em: https://github.com/dotnet/aspnetcore/blob/main/LICENSE.txt. Acesso em: 12 out. 2026.

[16] MICROSOFT. **C#**. Extensão para o Visual Studio Code. Visual Studio Marketplace. Disponível em: https://marketplace.visualstudio.com/items?itemName=ms-dotnettools.csharp. Acesso em: 12 out. 2026.

[17] MICROSOFT. **C# Dev Kit**. Visual Studio Marketplace. Disponível em: https://marketplace.visualstudio.com/items?itemName=ms-dotnettools.csdevkit. Acesso em: 12 out. 2026.

[18] MICROSOFT. **C# Extension for Visual Studio Code**: license terms. GitHub. Disponível em: https://github.com/dotnet/vscode-csharp/blob/main/RuntimeLicenses/license.txt. Acesso em: 12 out. 2026.

[19] MICROSOFT. **Introduction to Identity on ASP.NET Core**. Disponível em: https://learn.microsoft.com/en-us/aspnet/core/security/authentication/identity. Acesso em: 12 out. 2026.

[20] MICROSOFT. **Migrations Overview**. Entity Framework Core. Disponível em: https://learn.microsoft.com/en-us/ef/core/managing-schemas/migrations/. Acesso em: 12 out. 2026.

[21] MICROSOFT. **runtime/LICENSE.TXT**. GitHub. Disponível em: https://github.com/dotnet/runtime/blob/main/LICENSE.TXT. Acesso em: 12 out. 2026.

[22] MICROSOFT. **The C# type system**. Disponível em: https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/types/. Acesso em: 12 out. 2026.

[23] MICROSOFT. **Visual Studio Code FAQ**. Disponível em: https://code.visualstudio.com/docs/supporting/faq. Acesso em: 12 out. 2026.

[24] MICROSOFT. **Visual Studio Community**. Disponível em: https://visualstudio.microsoft.com/vs/community/. Acesso em: 12 out. 2026.

[25] OPENJDK. **GNU General Public License, version 2, with the Classpath Exception**. Disponível em: https://openjdk.org/legal/gplv2+ce.html. Acesso em: 12 out. 2026.

[26] PMI - PROJECT MANAGEMENT INSTITUTE. **A Guide to the Project Management Body of Knowledge (PMBOK Guide)** and The Standard for Project Management. 7. ed. Newtown Square: Project Management Institute, 2021.

[27] PMI - PROJECT MANAGEMENT INSTITUTE. **Um Guia do Conhecimento em Gerenciamento de Projetos (Guia PMBOK)**. 6. ed. Newtown Square: Project Management Institute, 2017.

[28] PYTHON SOFTWARE FOUNDATION. **History and License**. Disponível em: https://docs.python.org/3/license.html. Acesso em: 12 out. 2026.

[29] PYTHON SOFTWARE FOUNDATION. **Status of Python versions**. Python Developer's Guide. Disponível em: https://devguide.python.org/versions/. Acesso em: 12 out. 2026.

[30] PYTHON SOFTWARE FOUNDATION. **typing: Support for type hints**. Disponível em: https://docs.p

ython.org/3/library/typing.html. Acesso em: 12 out. 2026.

[31] SAATY, T. L. **How to Make a Decision: The Analytic Hierarchy Process**. Texto de palestra de abertura (*keynote address*). [S.l.: s.n.], [s.d.]. Disponível em: https://www.isahp-org.creativedecisions.net/uploads/1-how-to-make-a-decision-the-analytic-hierarchy-process-t-l-saaty.pdf. Acesso em: 12 out. 2026.

[32] SPRING. **Authorize HttpServletRequests**. Spring Security. Disponível em: https://docs.spring.io/spring-security/reference/servlet/authorization/authorize-http-requests.html. Acesso em: 12 out. 2026.

[33] SPRING. **Database Initialization**. Spring Boot. Disponível em: https://docs.spring.io/spring-boot/how-to/data-initialization.html. Acesso em: 12 out. 2026.

[34] SPRING. **JDBC Authentication**. Spring Security. Disponível em: https://docs.spring.io/spring-security/reference/servlet/authentication/passwords/jdbc.html. Acesso em: 12 out. 2026.

[35] SPRING. **spring-boot/LICENSE.txt**. GitHub. Disponível em: https://github.com/spring-projects/spring-boot/blob/main/LICENSE.txt. Acesso em: 12 out. 2026.

[36] SPRING. **Spring Data JPA**. Disponível em: https://docs.spring.io/spring-data/jpa/reference/. Acesso em: 12 out. 2026.

[37] SPRING. **Spring Security Reference**. Disponível em: https://docs.spring.io/spring-security/reference/index.html. Acesso em: 12 out. 2026.

[38] SPRING. **Support Policy**. Disponível em: https://spring.io/support-policy. Acesso em: 12 out. 2026.

[39] STACK OVERFLOW. **2025 Developer Survey: Technology**. Disponível em: https://survey.stackoverflow.co/2025/technology/. Acesso em: 12 out. 2026.

[40] THE GO AUTHORS. **Go 1 and the Future of Go Programs**. Disponível em: https://go.dev/doc/go1compat. Acesso em: 12 out. 2026.

[41] THE GO AUTHORS. **Go License**. Disponível em: https://go.dev/LICENSE. Acesso em: 12 out. 2026.

[42] THE GO AUTHORS. **Release History**. Disponível em: https://go.dev/doc/devel/release. Acesso em: 12 out. 2026.

[43] THE GO AUTHORS. **The Go Programming Language Specification**. Disponível em: https://go.dev/ref/spec. Acesso em: 12 out. 2026.

[44] UNIVERSIDADE DE SÃO PAULO. **ACH2003 - Computação Orientada a Objetos**. JupiterWeb. Disponível em: https://uspdigital.usp.br/jupiterweb/obterDisciplina?sgldis=ACH2003. Acesso em: 12 out. 2026.

[45] UNIVERSIDADE DE SÃO PAULO. **Grade curricular: Bacharelado em Sistemas de Informação**. JupiterWeb. Currículo com início em 1º jan. 2027. Disponível em: https://uspdigital.usp.br/jupiterweb/listarGradeCurricular?codcg=86&codcur=86200&codhab=202&tipo=N. Acesso em: 12 out. 2026.

[46] UNIVERSIDADE DE SÃO PAULO. Coordenadoria de Administração Geral. **Diretrizes Orçamentárias e Revisão do Planejamento Plurianual 2026**. São Paulo: USP, 2025. Disponível em: https://sites.usp.br/codage/wp-content/uploads/sites/264/2026/01/2026_Diretrizes-Orcamentarias.pdf. Acesso em: 12 out. 2026.

[47] VARGAS, R. V. **Using the Analytic Hierarchy Process (AHP) to Select and Prioritize Projects in a Portfolio**. Trabalho apresentado no PMI Global Congress 2010 - North America, Washington, DC, 2010. Disponível em: https://ricardo-vargas.com/articles/analytic-hierarchy-process/. Acesso em: 12 out. 2026.